

PROMO X- SEMESTRE Y- FORMATION



Nom de l'Enseignant :
Haby DIALLO

Cours :
Administration de base de données

Séquence 2



CHAPITRE 2 : LES MODÈLES AVANCÉS DE BASES DE DONNÉES

Objectifs spécifiques :

A la fin de ce chapitre, l'étudiant doit être capable de:

- Identifier les différents modèles de base de données
- Connaître les limites des SGBD
- Connaître les caractéristiques des DBOO
- Connaître les concepts objets et le langage OQL
- Se familiariser avec le langage SQL3





Titre: CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

10 ans
2013 - 2023

CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Introduction: Limites des SGBDR**

- **Force du Modèle Relationnel**

- Le modèle de données est très simple (tables, relations...) et donc facile à comprendre pour les utilisateurs.
 - Le modèle repose sur une base formellement définie (normalisation, algèbre, ...), ce qui a permis de définir des langages de manipulation de données (LMD) standardisés tels que(QUEL, SQL)
 - Efficace pour les applications de gestion classique



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

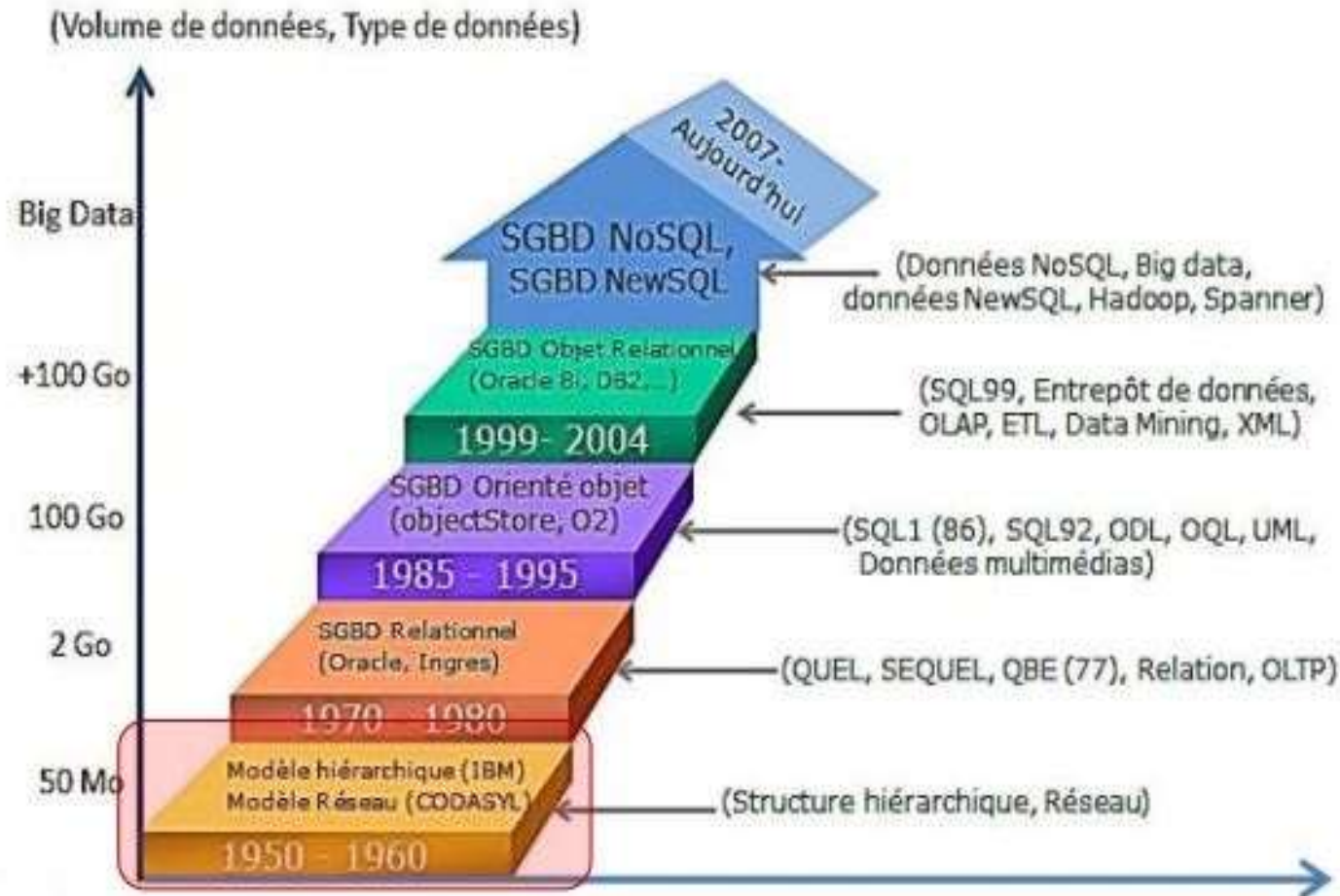
- **Introduction: Limites des SGBDR (suite)**

- **Faiblesses du Modèle Relationnel**

- Modèle de données simple : difficile de modéliser des structures complexes :
 - pas d'attribut complexe, ni multivalué
 - Données alphanumériques uniquement (pas d'images, de sons, de vidéo, ...)
 - un seul type de lien (clé étrangère) : jointure indispensable
 - le système de types figé (non extensible)
 - Les objets sont éclatés en plusieurs relations, ce qui multiplie les jointures dans les requêtes des utilisateurs
- L'incompatibilité des LMD relationnels et des langages de programmation
- Le développement d'applications est très lent et les applications sont difficilement maintenables.
- Mal adapté à :
 - de nouvelles applications : aide à la décision, conception, géographie, bureautique, multimédia, ...
 - de nouvelles techniques : IHM, programmation orientée objet, programmation en logique, architectures réparties,
 - ...

CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Introduction: évolution des SGBD

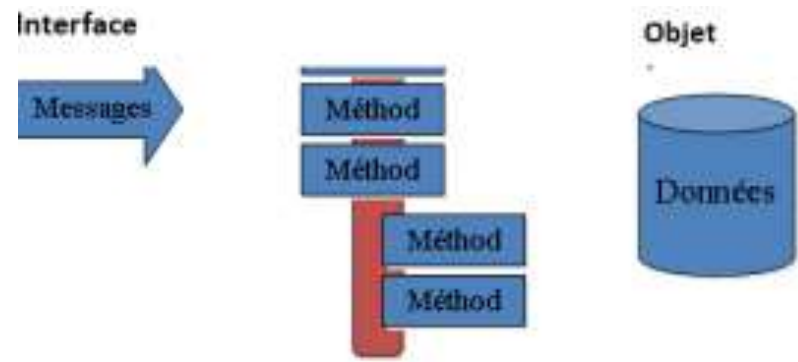




CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Définition d'un objet**

- Entité informatique complète identifiable caractérisée par son état et un ensemble de méthodes.
- Associe traitements (méthodes) et données dans une même entité.
- L'interface est le seul moyen d'accéder à l'objet, par l'envoi de messages.



- Abstraction informatique d'une entité du monde réel caractérisée par une identité, un état et un comportement.
- Un objet est l'instance d'une classe, et une classe, est un type de données abstrait, caractérisé par des propriétés (ses attributs et ses méthodes) communes à des objets, qui permet de créer des objets possédant ces propriétés.



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Modèle orienté objet**
 - Premier SGBD objet : 1983 (Gemstone)
 - Le **modèle Orienté objet** est une ensemble de méthodologies et d'outils pour concevoir et réaliser des logiciels structurés et réutilisables, par composition d'éléments indépendants.
 - Les **SGBDOO** possèdent certaines caractéristiques propres au paradigme objet qui les distinguent des autres applications.
 - Ces concepts sont : l'identité d'objet appelé OID (Object Identifier), l'héritage, l'encapsulation, les classes et types, l'agrégation, le polymorphisme et l'extensibilité.
 - Objectif : productivité des programmeurs
 - Moyen : réutilisation
 - Concepts essentiels:
 - Objet encapsulé
 - interface visible : opérations (méthodes)
 - implémentation cachée : structure et code
 - Héritage
 - Langages de programmation OO :
 - C++, Java, Smalltalk, ...
 - Produits
 - O2, ObjectStore, Ontos, Objectivity, Jasmine, Versant
 - Niches technologiques
 - réseau, CAO, SIG, Gestion de données Techniques
 - pas de transactionnel lourd



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Les concepts de base d'un objet**

- **L'identité** : L'objet possède une identité, qui permet de le distinguer des autres objets, indépendamment de son état. On construit généralement cette identité grâce à un identifiant découlant naturellement du problème (par exemple un produit pourra être repéré par un code, une voiture par un numéro de série, ...)
- **Les attributs** : Il s'agit des données caractérisant l'objet. Ce sont des variables stockant des informations d'état de l'objet
- **Les méthodes (appelées parfois fonctions membres)** : Les méthodes d'un objet caractérisent son comportement, c'est-à-dire l'ensemble des actions (appelées opérations) que l'objet est à même de réaliser. Ces opérations permettent de faire réagir l'objet aux sollicitations extérieures (ou d'agir sur les autres objets).
- De plus, les opérations sont étroitement liées aux attributs, car leurs actions peuvent dépendre des valeurs des attributs, ou bien les modifier



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

Exemple:

Rectangle
Longueur=3 Largeur = 2
Origine = Point Surface() Perimetre() DeplacerA(p)

Point
x=0 y=1 Deplacer(x,y)

← Identité

← Etat

← Comportement



- **Règle d'or d'un SGBD objet**

- Pour être qualifié d'objet, un SGBD doit respecter treize règles:

- 1. Données persistantes**

- Les données persistantes et non persistantes (temporaires) sont manipulées de la même façon par un programme

- 2. Grande quantité de données**

- Techniques de regroupement physique, d'indexation, d'optimisation de requêtes et de gestion de cache

- 3. Fiabilité des données**

- Transactions et privilèges



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Règle d'or d'un SGBD objet(suite)**

- Pour être qualifié d'objet, un SGBD doit respecter treize règles:

4. Partages de données

- multi-utilisateur
- mécanismes de verrous

5. Facilité d'interrogation

- Langage de requêtes.
- Le résultat d'une requête n'est pas forcément un objet d'une classe existante.
- Le langage offre en outre la puissance d'un langage de Programmation

6. Encapsulation

- Les données sont accessibles par des méthodes à différents degrés (visibilité)

7. Objets composites

- Les structures de données à définir et à manipuler peuvent être complexes

- **Règle d'or d'un SGBD objet(suite)**

- Pour être qualifié d'objet, un SGBD doit respecter treize règles:

8. Identificateur d'objet

- Identifier d'une manière unique dans la base de données n'importe quel objet de cette base
- Accès aux objets directement ou par des liens inter-objets via les OID
- Objets partagés

9. Classes, types et méthodes

- Types abstraits de données
- Concepts de classes et de méthodes de l'approche objet



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Règle d'or d'un SGBD objet(suite)**

- Pour être qualifié d'objet, un SGBD doit respecter treize règles:

10. Héritage

- Permet la réutilisation des types de données, car il permet la définition d'objets génériques (tables, types de données abstraits) pouvant être spécialisés suivant les utilisations.

11. Surcharge et liaison dynamique

- Les méthodes peuvent être surchargées. La liaison dynamique est la capacité d'établir la correspondance entre le nom d'une méthode et son implantation lors de l'exécution et non lors de la compilation

- **Règle d'or d'un SGBD objet(suite)**

- Pour être qualifié d'objet, un SGBD doit respecter treize règles:

12. Langage de programmation complet

- Le système dispose d'un langage de programmation qui ne nécessite pas d'opérateur externe pour écrire une application

13. Extensibilité

- Le système autorise l'ajout dynamique de nouveaux types abstraits de donnée, de nouvelles classes, de nouvelles méthodes, etc.

- **SGDBOO ou SGBDRO ?**

- les **SGBDOO** sont plus “propres” du point de vue objet et les mieux adaptés pour traiter les objets mais ils sont complètement absents du monde professionnel.
- les **SGBDRO** sont basés sur des SGBD robustes et éprouvés répandus dans le monde professionnel mais qui ne sont pas prévus pour gérer l'objet.
- Jusqu'à présent, les améliorations qui ont été développées pour les SGBD se sont fondus dans les SGBD existants.
- Dans le monde professionnel, les concepteurs et les utilisateurs de bases de données ne sont pas prêts à remettre en cause leurs savoirs et à redévelopper toutes leurs applications sur de nouveaux systèmes.



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **SGDBOO ou SGBDRO ?**

- On va privilégier les **SGBDRO**
- Le modèle objet-relationnel est fondé sur l'idée d'extension du modèle relationnel avec les concepts essentiels de l'objet.
- Le cœur du modèle reste donc conforme au relationnel, mais l'on ajoute les concepts clés de l'objet sous une forme particulière pour faciliter l'intégration des deux modèles.
- Le modèle objet-relationnel conserve :
 - **Les notions de base du relationnel** : domaine, table, attribut, clé et contrainte référentielle
 - **Les concepts de l'objet**: de type utilisateur avec structure éventuellement privée et opérations encapsulant, de collections supportant des attributs multi values (structure, liste, tableau, ensemble, etc.), d'héritage sur relations et types, et d'identifiants d'objets permettant les références inter tables. Les domaines peuvent dorénavant être des types utilisateurs.

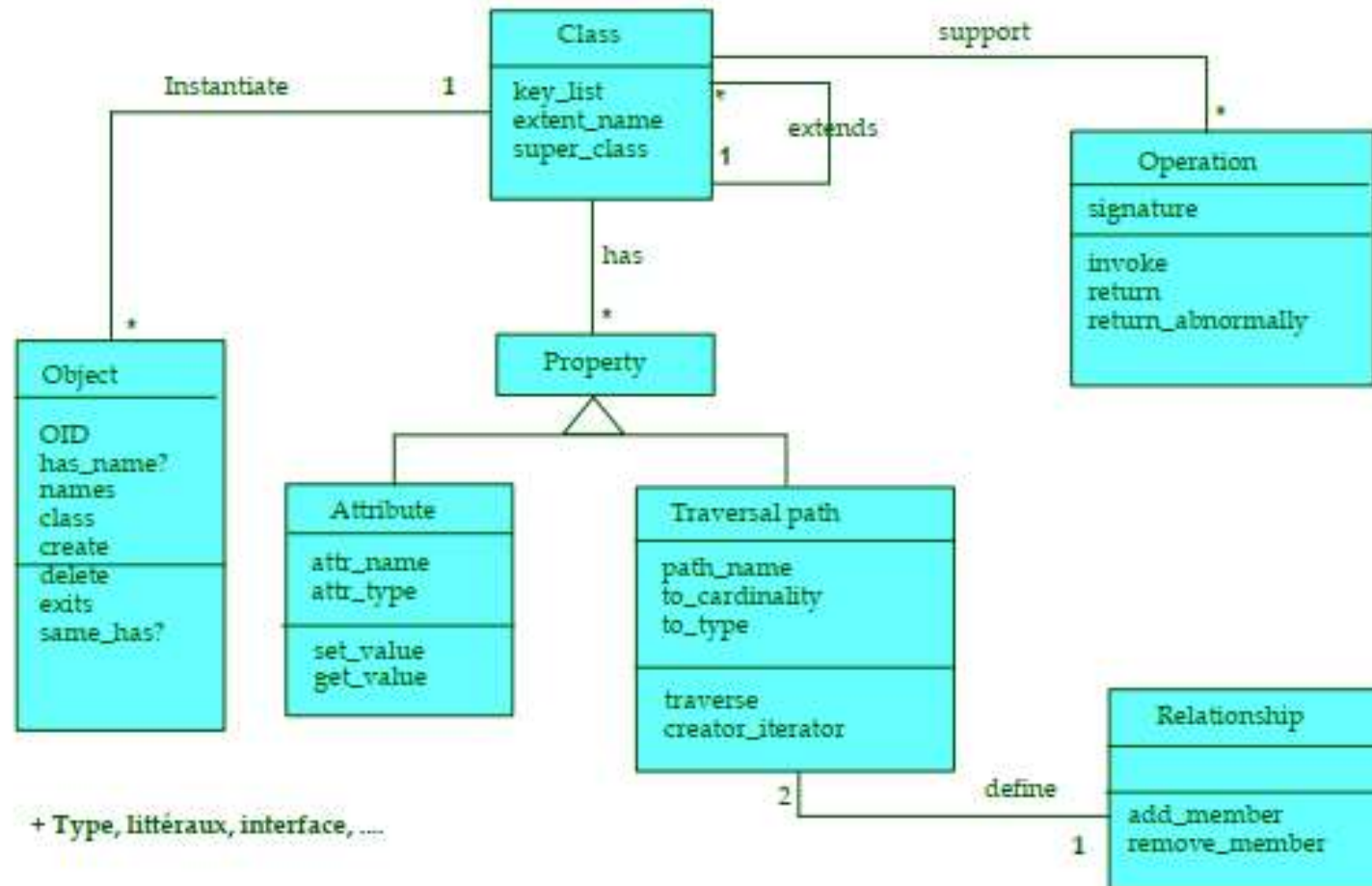
- **Langages**

- La force des SGBD relationnels vient de la normalisation des SQL, ESQL et SQL/CLI
- **Quels langages pour les SGBD OO ?**
 - diversité des modèles et des langages
 - Effort de normalisation
 - OQL de l'ODMG ou SQL3 de l'ANSI ?
 - liens avec les langages de programmation OO
 - impact sur le développement d'applications



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

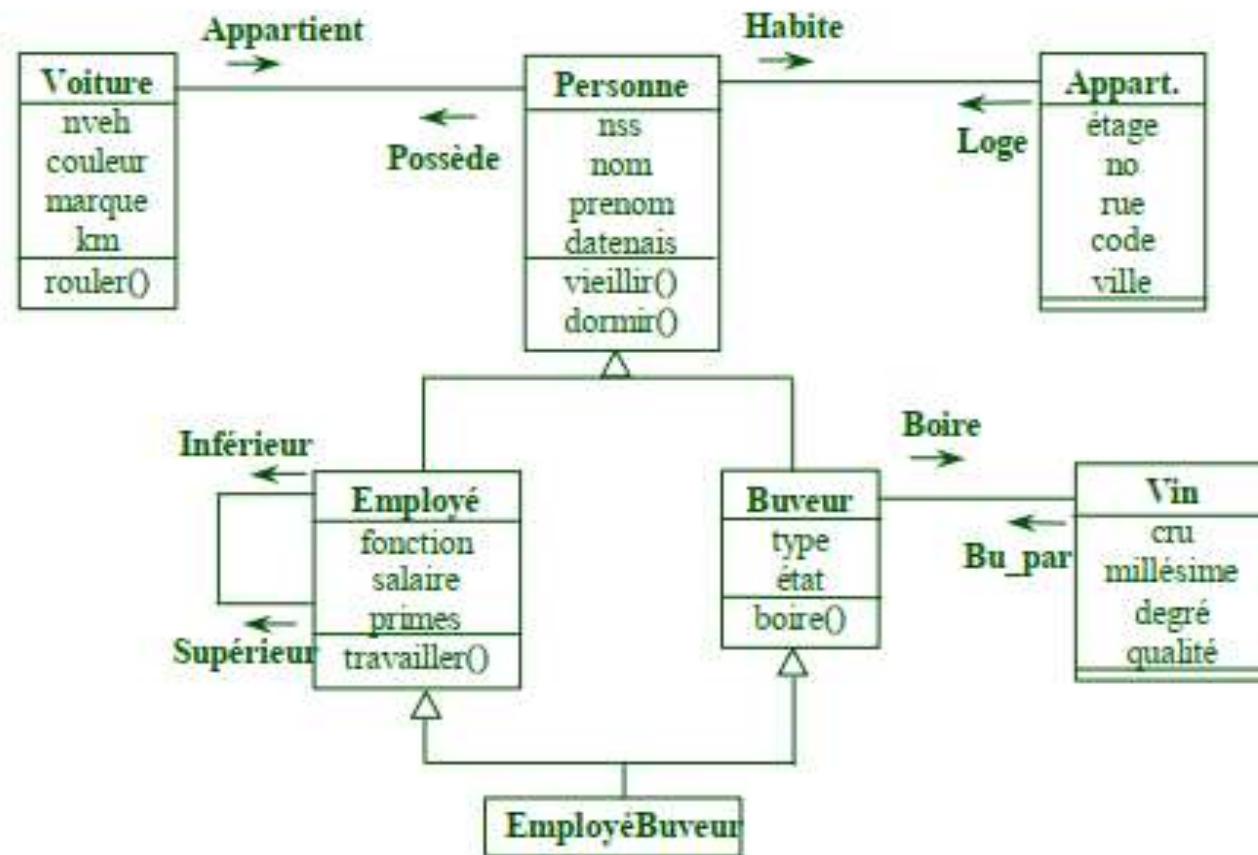
- Méta-modèle du modèle ODMG





CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Exemple de base





CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Exemple de définition**

Interface Personne { // interface abstraite pour implémentation dans classe

attribute string nss ;

attribute string nom ;

attribute string prenom ;

attribute date datenaissance;

relationship Appart habite inverse Appart::loge; // relationship

relationship Voiture Possede inverse Voiture::Appartient;

short vieillir();

void dormir()

short age();

};

- **Exemple (suite)**

```
class Employé : Personne(extent Employé key nss)  
//classe avec extension de personne +  
{  
attribute enum fonct{ingénieur, secrétaire, analyste, programmeur} fonction;  
attribute float salaire ;  
attribute list<float> primes ; //attribut multi-valué  
relationship Employé inferieur inverse supérieur;  
relationship Employé supérieur inverse inférieur;  
void travailler();  
};
```



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Le langage OQL(Object Query Language)**
 - Permettre un accès facile à une base objet
 - via un langage interactif autonome
 - par intégration comme extension d'un langage de programmation hôte, comme **C++** ou **Smalltalk** ou **Java**
 - Offrir un accès non procédural
 - permettre des optimisations automatiques (ordonnancement, index,...)
 - garder une syntaxe proche de **SQL 92**
 - Rester conforme au modèle de l'**ODMG**
 - application d'opérateurs aux collections extensions ou imbriquées
 - permettre de créer des résultats littéraux, objets, collections, ...
 - Supporter des mises à jour limitées via les méthodes

- Format général d'une requête OQL ^

***SELECT** liste d'expressions*

***FROM** liste d'une ou plusieurs déclarations de variables.*

***WHERE** condition C de sélection*

- Une variable est déclarée en indiquant : '
 - Une expression dont la valeur a un type collection, par exemple **set** ou **bag**
(expression indique l'extension d'une classe, par exemple Films.
Analogie avec une relation dans une requête SQL)
 - Le nom de la variable



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Format général d'une requête OQL(suite)**
 - **Notation “.”**
 - Soit **o** un objet d'une classe **C**.
 - Si **p** est un attribut, **o.p** est la valeur de **p** pour **o**.
 - Si **p** est une relation, **o.p** est l'objet ou la collection d'objets reliées à **o** par **p**.
 - Si **p** est une méthode (éventuellement avec paramètres **a₁, . . . a_k**), **o.p(...)** est le résultat de l'application de **p** à **a**.
 - Exemple: La déclaration de la méthode **NomsActeurs** de la classe **Film** est:
void NomsActeurs(out Set<String>) ;
 - si **MonFilm** est un objet de la classe **Film**, l'expression:
MonFilm.NomsActeurs(mesStars) ne renvoi pas de valeur, mais, comme effet de bord, pose la valeur de la variable output **mesStars** de la méthode comme étant un ensemble de noms d'acteurs.



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Exemple d'une BD objet pour illustrer OQL

```
class Film(extent Films key (titre, année))
{
    attribute string titre ;
    attribute integer année ;
    attribute integer longueur ;
    attribute enum couleurs
    { couleur, noir&blanc} SorteFilm;
    relationship Set<Star> acteurs inverse Star : :JoueDans;
    relationship Studio appartient-à inverse Studio : :possède;
    float longueurHeures() raises(pasLongueurTrouvée);
    void NomsActeurs(out Set<String>) ;
    void autresFilms (in Star, out Set<Film>) raises(PasActeur);} ;

class Star (extent Stars key nom)
{
    attribute string nom ;
    attribute Struct Adr {string rue, string ville} adresse;
    relationship Set<Film> JoueDans inverse Film : :acteurs;
};

class Studio (extent Studios key nom)
{
    attribute string nom ;
    attribute string adresse;
    relationship Set<Film> :possède inverse Film : :appartient-à;
};
```



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Exemples de Requêtes OQL

```
SELECT m.année
```

```
FROM Films m
```

```
WHERE m.titre = "Autant en emporte le vent"
```

- Ici, **Films m** est une déclaration d'une variable **m** qui a sa valeur dans l'extension **Films** de la classe dont le nom est **Film**, i.e. : **valeur(m) ∈ Films**.

```
SELECT s.nom
```

```
FROM Films m, m.acteurs s
```

```
WHERE m.titre = "Casablanca"
```

- Dans l'évaluation, tous les couples **(m,s)** tels que **m** est un film et **s** est un acteur de **m** (selon la relation acteurs de Film) sont considérés :

```
FOR chaque m dans FILMS DO
```

```
FOR chaque s dans m.acteurs DO
```

```
IF m.titre = "Casablanca" THEN ajouter s.nom au multi-ensemble resultat.
```

La condition après le **WHERE** limite l'espace de recherche pour m aux films dont le titre est Casablanca".



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Exemples de Requêtes OQL (suite)

```
SELECT DISTINCT s.nom
```

```
FROM Films m, m.acteurs s
```

```
WHERE m.appartient-`a.nom = "Disney"
```

- Comme en **SQL**, le mot clé **DISTINCT** élimine la répétitions dans le multi-ensemble (bag) **resultat**.
- Liste des noms des films du studio Disney, ordonnés par leur longueur ; si deux films ont la même longueur, trier selon l'ordre alphabétique des titres. '

```
SELECT m
```

```
FROM Films m
```

```
WHERE m.appartient-`a.nom = "Disney"
```

```
ORDER BY m.longueur, m.titre
```

- **Exemples de Requêtes OQL (suite)**

- On veut l'ensemble des couples de stars qui vivent a la même adresse : ^

SELECT DISTINCT Struct(star1 :s1, star2 : s2)

FROM Stars s1, Stars s2

WHERE s1.adresse = s2.adresse AND s1.nom < s2.Nom

- Pour chaque couple qui passe le test on produit un record, avec 2 champs, nommés ***star1 et star2***. Le type de chaque champ est la classe **Star**.

Le type du résultat final de cette requête est :

Set<Struct{star1 : Star, star2 : Star}>.



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Exemples de Requêtes OQL (suite)

- Utilisation de “sous-requetes” ^

- Une autre façon de chercher les noms des acteurs des films du studio Disney :

```
SELECT DISTINCT s.nom
```

```
FROM (SELECT m
```

```
FROM Films m
```

```
WHERE m.appartient-`a.nom = "Disney") f, f.acteurs s
```

- La variable **m** prend valeur dans la collections des films du studio Disney, qui est le résultat de la sous-requete :

```
SELECT m
```

```
FROM Films m
```

```
WHERE m.appartient-`a.nom = "Disney"
```

- La variable **s** prend valeur dans la collections des acteurs du film de Disney **f**.



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Le relationnel-objet sur Oracle**

- **Définition de Types**

- Oracle permet de définir des types du style objet. Syntaxe : ´

CREATE TYPE t AS OBJECT (liste d'attributes et methodes);

/ (Le slash a la fin sert pour faire traiter par Oracle la définition de type)

Exemple. Définition d'un type pour représenter un point par ses coordonnees : ´

CREATE TYPE PointType AS OBJECT (

x NUMBER,

y NUMBER

);

/



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Le relationnel-objet sur Oracle(suite)
 - Définition de Types(suite)
 - Un type objet peut être utilisé pour déclarer d'autres types objet ou relations (au sens traditionnel, mais pas en 1FN), par ex. on peut définir un type pour les lignes (géométriques) finies en indiquant les 2 extrémités : '

```
CREATE TYPE LineType AS OBJECT (  
    end1 PointType,  
    end2 PointType );  
  
/
```

- On peut alors créer une relation (presque traditionnelle) qui est un ensemble de lignes, où chaque ligne a un OID :

```
CREATE TABLE Lines (  
  
    lineID INT,  
    line LineType);
```

N.B: Ici, on a un attribut (LyneType) qui est de type non-atomique.

- Le relationnel-objet sur Oracle(suite)
 - Elimination de Types
 - Pour supprimer un type comme **LineType** :

DROP TYPE Linetype;

N.B. On doit au préalable supprimer toutes les tables et les autres types qui utilisent ce type (dans notre exemple : la table **Lines**).



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Le relationnel-objet sur Oracle(suite)
 - Donner des valeurs aux objets

*INSERT INTO TYPEOBJET
VALUE(liste des valeurs);*

- Exemple: Construire une ligne de la table **Lines** ayant comme identifiant 27, qui part du point(0,0) et arrive au point(3,4) :

```
INSERT INTO Lines  
VALUES(27, LineType(  
                                PointType(0.0, 0.0),  
                                PointType(3.0, 4.0)  
                                )  
);
```

N.B: la création de valeurs pour le type **Lines** a aussi entraîné la création de 2 valeurs pour le type **PointType** et 1 valeur pour le type **LineType**.



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Le relationnel-objet sur Oracle(suite)
 - Déclarations de méthodes
 - Dans la déclaration de type on peut aussi inclure la déclaration de méthodes, à l'aide de **MEMBER FUNCTION** ou **MEMBER PROCEDURE** en spécifiant le type des entrées et sortie.
 - Dans notre exemple précédant nous allons ajouter une fonction **length()** au type **LineType**, qui donne la longueur d'une ligne et la multiplie par un facteur.

```
CREATE TYPE LineType AS OBJECT (  
    end1 PointType,  
    end2 PointType,  
    MEMBER FUNCTION length(scale IN NUMBER) RETURN NUMBER,  
    PRAGMA RESTRICT_REFERENCES(length, WNDS)  
);  
/
```

NB: -La dernière instruction (PRAGMA...) dit que la méthode ne change pas la BD.
-(WNDS = write no database state).



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Le relationnel-objet sur Oracle(suite)
 - Définition du code d'une méthode
 - Le code de la méthode est donnée à part, par l'instruction **CREATE TYPE**
 - Dans notre exemple:

```
CREATE TYPE BODY LineType AS
```

```
MEMBER FUNCTION length(scale NUMBER) RETURN NUMBER IS
```

```
BEGIN
```

```
RETURN scale *SQRT((SELF.end1.x-SELF.end2.x)*(SELF.end1.x-
```

```
SELF.(SELF.end1.y-SELF.end2.y)*(SELF.end1.y-SELF.);
```

```
END;
```

```
END;
```

```
/
```

- **NB:** La variable **SELF** dans la definition de la méthode indique la valeur courante.

- Le relationnel-objet sur Oracle(suite)
 - Quelques exemples de requête relationnelle-objet
 - Calcul du double de la longueur des chaque ligne :

```
SELECT lineID, L1.line.length(2.0)  
FROM Lines L1;
```

 - Pour la requête, on utilise la méthode leghth().
 - Un alias est utilisé (ici, L1) pour accéder au champ avec la notation ".".
 - En fait, **line** tout seul **et Lines.line** ne marchent pas .
 - Coordonnees de la première extrémité de chaque ligne :

```
SELECT L1.line.end1.x, L1.line.end1.y  
FROM Lines L1;
```

- Le relationnel-objet sur Oracle(suite)
 - Quelques exemples de requête relationnelle-objet
 - Types Références
 - Si **t** est un type, **REF t** est le type des références (**OID**) à des valeurs de type **t**.

Exemple : définition d'un schéma de table avec **REF**.

Une relation **Lines2** dont les lignes sont des couples de références à des points :

```
CREATE TABLE Lines2 (  
    end1 REF PointType,  
    end2 REF PointType  
);
```



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Le relationnel-objet sur Oracle(suite)
 - Quelques exemples de requête relationnelle-objet
 - Types Références
 - Exemple, suite : définition du contenu d'une table avec REF.
 - Supposons qu'on a déjà une table **Points** dont les valeurs sont de type **PointType**.
 - On remplit la table **Lines2** avec des valeurs qui sont toutes les lignes dont les extrémités (gauche et droite) sont des éléments de la table **Points** :

INSERT INTO Lines2

SELECT REF(pp), REF(qq)

FROM Points pp, Points qq

WHERE pp.x < qq.x;

- Attention : On ne peut pas créer un objet qui n'est pas déjà dans une relation et le réfère
- Mauvais exemple:

INSERT INTO Lines2 VALUES(REF(PointType(1,2)), REF(PointType(3,4)));

- Les objets PointType(1,2) et PointType(1,2) n'existent pas

CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Le relationnel-objet sur Oracle(suite)
 - Quelques exemples de requête relationnelle-objet
 - Types Références
 - Dans notre exemple : Calcul des coordonnées **x** des extrémités de chaque ligne élément de **Lines2** : ***SELECT L1.end1.x, L1.end2.x FROM Lines2 L1;***



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Le relationnel-objet sur Oracle(suite)
 - Quelques exemples de requête relationnelle-objet
 - Relations imbriquées, déclaration du schéma
 - La valeur d'un attribut d'une table peut être une relation.
 - Définition du schéma d'une table dont les éléments sont des polygones, tout polygone étant une table (de points) lui-même.

```
CREATE TYPE PolygonType AS TABLE OF PointType; /
```

```
CREATE TABLE Polygons (  
    name VARCHAR2(20),  
    points PolygonType)
```

```
NESTED TABLE points STORE AS PointsTable;
```

- NB: **NESTED** indique que les sous-tables présentant les polygones ne sont pas stockées directement comme valeurs de l'attribut points, mais dans une autre table dont le nom est

- Le relationnel-objet sur Oracle(suite)
 - Quelques exemples de requête relationnelle-objet
 - Relations imbriquées, remplissage des tables
 - La valeur de chaque relation de niveau inférieur est représentée par une liste de valeurs du type approprié (**PolygoneType** dans notre exemple).
 - Exemple d'insertion d'un polygone qui est un carre' :

```
INSERT INTO Polygons VALUES(  
    'square', PolygonType(PointType(0.0, 0.0), PointType(0.0,  
    PointType(1.0, 0.0), PointType(1.0,));
```



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- Le relationnel-objet sur Oracle(suite)s
 - Quelques exemples de requête relationnelle-objet
 - Relations imbriquées, exemples de requêtes

- Les points du carre' :

SELECT points

FROM Polygons

WHERE name = 'square';

- Le point du carre qui sont sur la diagonale ($x=y$):

SELECT ss.x

FROM THE(SELECT points

FROM Polygons

WHERE name = 'square') ss

WHERE ss.x = ss.y;

NB: (sous-requête, relation du niveau inférieur dans le **FROM** avec le mot-clé **THE** et utilisation de la variable **ss** pour la nommer.)

- **SQL3**

- Nouvelle norme SQL de l'ANSI
 - Multiples facettes :
 - Un langage de définition de types
 - Un langage de programmation
 - Un langage de requêtes
 - Un langage temporel
 - ...
 - Données complexes, objet-relationnel
 - Adeptes :
 - OO : Illustra, UniSQL, Versant
 - Relationnel : Ingres, Oracle, DB2, Informix

- **SQL3 : concepts objet**

- Extensibilité des types de données
 - Définition de types abstraits
 - Possibilité de types avec ou sans OID
- Support d'objets complexes
 - Constructeurs de types (tuples, set, list, ...)
 - Identité d'objet et partage d'objet
- Héritage
 - Définition de sous-types
 - Définition de sous-tables



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **SQL3 : définition de types**

- Un type avec référence

```
CREATE TYPE WITH OID TELEPHONE
```

```
(pays VARCHAR, zone VARCHAR, num number(10),description CHAR(20))
```

- Un type sans référence

```
CREATE TYPE PERSONNE(nss NUMBER, nom VARCHAR, prénoms  
LIST(VARCHAR), tel  
SET(TELEPHONE))
```

- Un sous-type

```
CREATE TYPE EMPLOYE UNDER PERSONNE
```

```
(salaire NUMBER(10,2), ancienneté DATE)
```



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **SQL 3 : relations imbriquées**

- Une table peut posséder des attributs d'un type abstrait
- Un tuple peut contenir des références ou des valeurs complexes
- Possibilité d'utiliser un type prédéfini

CREATE TABLE Films OF Film ;

- Possibilité de définir un nouveau type

CREATE TABLE Acteurs OF NEW TYPE Acteur (nom VARCHAR, age NUMBER) ;

- Possibilité de définir des sous-tables

CREATE TABLE Films_Français UNDER Films (realisateurs VARCHAR)



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **SQL3 : utilisation des fonctions**

- Fonctions

```
SELECT      a.nom  
FROM        Bateau a, Bateau b  
WHERE       b.nom = 'Grand Bleu'  
and         distance(a.position,b.position) < 1
```

- Opérateurs

```
SELECT      a.nom  
FROM        Bateau a, Bateau b  
WHERE       b.nom='Grand Bleu'  
and         contenu(a.position, zone(b.position, 1))
```



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- SQL 3 : exemple de relations imbriquées

Films

N°	Budget	Réalisateur	Acteurs		Dépenses		
			Nom	Age	Num	Montant	Motif
2	4MF	Besson	Jean	55	1	2600	Bande
			Julie	40	2	8700	Hôtel
					3	15400	Caméra
5	5MF	Chouraki	Sylvie	45	5	3000	Bande
			Pierre	29	7	4000	Hôtel



CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **SQL 3 : accès navigationnel**

- Accès en relationnel

```
SELECT Films.n°, Films.budget  
FROM Acteurs, Films, Joue  
WHERE Acteurs.nom = Joue.acteur  
AND Joue.film = Films.n°  
AND Acteurs.nom = 'Besson'
```

- Accès en objet-relationnel

```
SELECT Films.n°, Films.budget  
FROM Films f  
WHERE f.acteurs.nom = 'Besson'
```

CHAPITRE 2 : LES BASES DE DONNÉES ORIENTÉES OBJET

- **Conclusion**

- **Objectifs atteints**

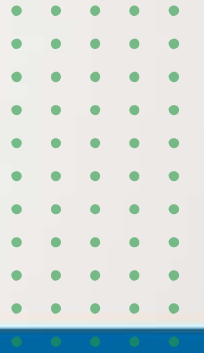
- meilleure représentation du monde réel
 - réutilisation
 - efficacité pour les applications nouvelles

- **MAIS**

- Les SGBDO ne sont pas adaptés à tout type d'application !
 - Méthodologies de conception incomplètes
 - normalisation de la structure, conception des méthodes
 - Compétition entre standards
 - Absence de théorie, formalisation
 - Vues
 - Evolution du schéma
 - Versions, temps
 - Migration difficile des SGBD classiques aux SGBDO



10^{ans}
de proximité



MERCI